

Lab Assignment 1

CMPE 146 Section 1

Professor Bhawandeep Singh Harsh

Andrew Bertch

Robert Trinh

Table of Contents

Exercise 1 – CCS and SDK	3
Exercise 2 – Hello world	4
Exercise 3 – Blink LED	5
Exercise 4 – Access MCU Info	6
Appendix A – MCU Source Code	10
HelloWorld.c	10
BlinkLED.c	10
Access_MCU_Info.c	12

Exercise 1 – CCS and SDK

In exercise 1, we were tasked with setting up the TI IDE, CCS (Code Compressor Studio), and the SDK for the MCU used in the TI LaunchPad we are using.

The 20.2.0 version of CCS was downloaded and installed with components for the MSPM0 Arm Cortex-M0+ microcontrollers was selected to be installed. An SDK, mspm0_sdk_2_05_01_00.exe, was downloaded and ran to install the SDK libraries for the MCU we are using. The final task completed in this exercise was to disable the auto check for update setting. This was to ensure that our software environment is stable for the development of programs throughout the semester.

No problems were encountered throughout the process of downloading, installing, and setting up the software and the development environment.

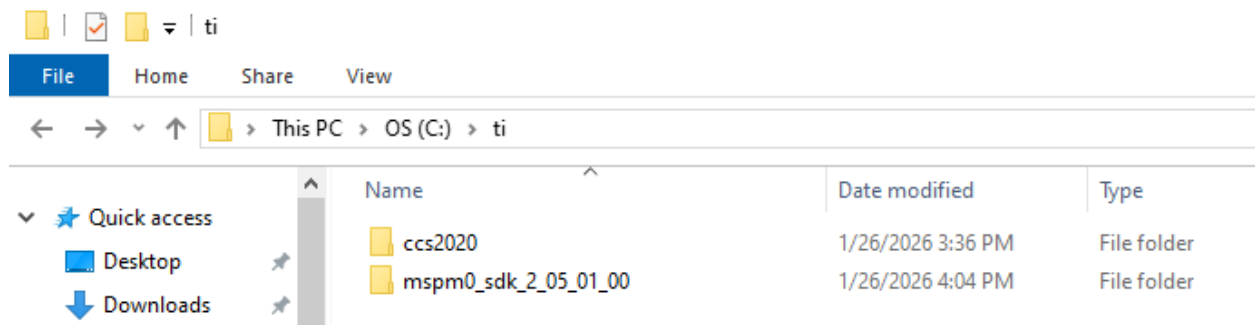


Figure 1: Screenshot of TI folder

Exercise 2 – Hello world

In exercise 2, we were tasked with creating a Hello world program to get familiar with CCS and SDK. We were also introduced with how CCS and the MCU interface with each other over a USB connection.

The LaunchPad was connected to the computer using the included USB cable and CCS was started. CCS automatically detected the LaunchPad that was connected to the computer when started. To simplify working with the LaunchPad and ARM Cortex Mo+, we were instructed on how to use projects in the SDK as a starting point instead of creating a empty project. For our Hello world program, we made use of the empty project from the SDK. After opening and renaming the project, a simple Hello world was programmed and a debug environment opened. Pressing F5 had CCS build and download the program to the LaunchPad and prepped the MCU to continue running the program. To have the program continue and have the MCU start executing, the continue button was pressed in CCS. The Hello world message was then seen in the CIO console in CCS.

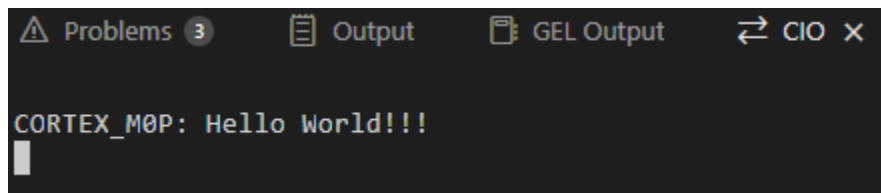


Figure 2: Screenshot of CIO

Exercise 3 – Blink LED

In exercise 3 we were tasked with importing an existing example blink project from the SDK. This introduced some of the basic IO functions that can be used by giving a completed project and introduced the use of delays for use in timing. We could also observe how to configure and initialize pins for use.

After importing the example project, we compiled and ran it. After observing the LED blinking, the delay value was changed to a quarter of its value. Recompiling and running the program it was simple to observe that the LED was now blinking faster with a shorter period.

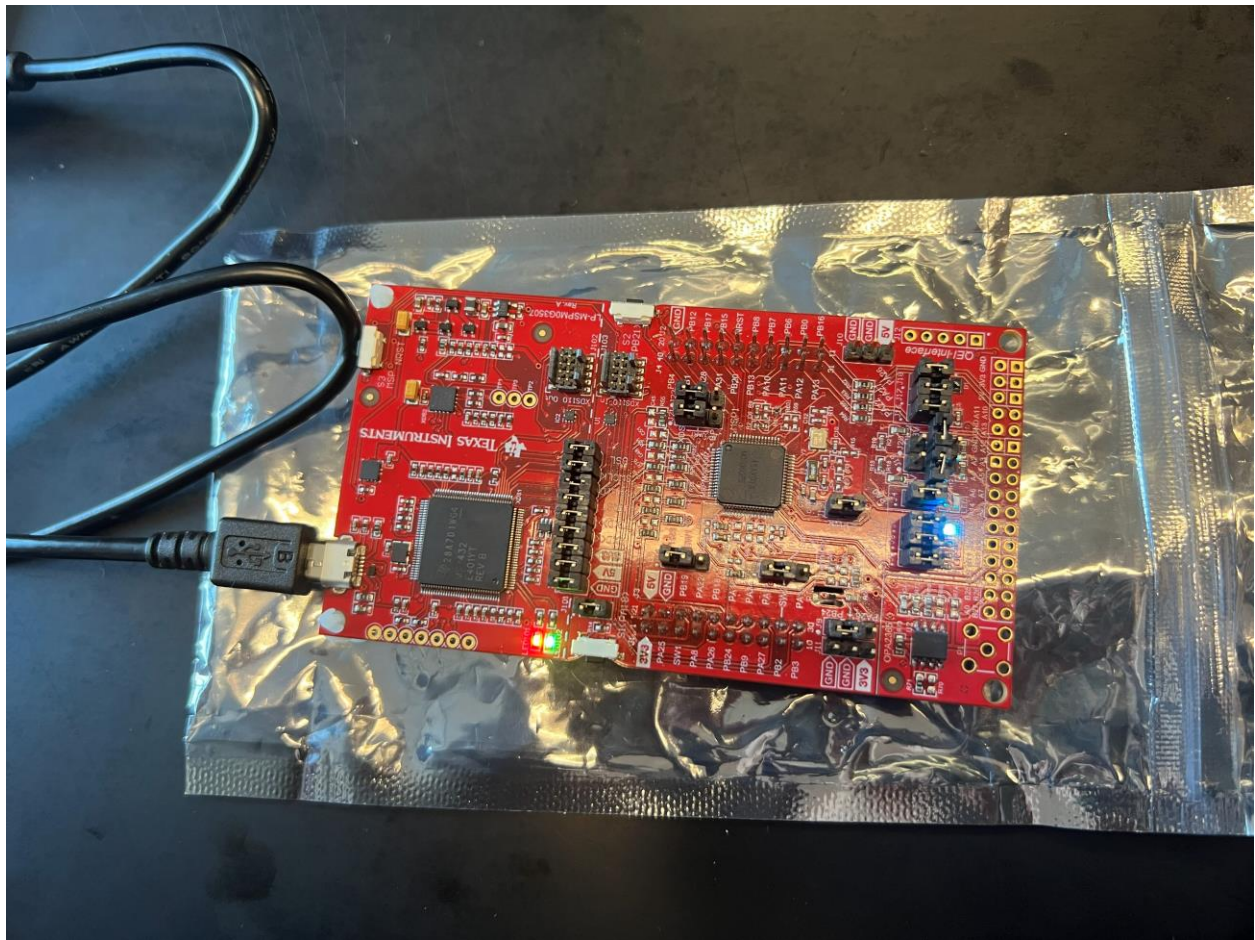


Figure 3: LED2 lit

A video of the LED blinking can be found at these links: [16000000](#), [4000000](#)

Exercise 4 – Access MCU Info

In exercise 4 we were tasked with writing a simple C program to access and print some of the device information of the TI Launchpad. This information was located in the device's FACTORY memory region.

To access the FACTORY memory region, we began by looking through the MCU technical reference to find the base memory address. It was found that the base memory address was located at 0x41C40000. The FACTORY memory region base address was required since we were not allowed to use any library functions or predefined data structures in DriverLib. Each FACTORYREGION register acronym was stored in a 2D character array. The FACTORY memory region base address was stored in a int and a pointer stored the base address plus the offset. To print the required information, register acronym, memory location, and register data, made use of 1 for loop with the register acronyms making use of another for loop. To access the data for each register, the pointer was set to the base memory address plus the register iterator times 4. The register iterator was multiplied by 4 since each data item was a 32-bit word.

Read and print code snippet:

```
for (int i = 0; i < 32; i++){
    for (int j = 0; j < strlen(FACTORYMEMORY_ACRONYM[i]); j++){
        printf("%c", FACTORYMEMORY_ACRONYM[i][j]);
    }
    ptr = (int *) (FACTORYMEMORY_OFFSET + i * 4);
    printf(": 0x%x -> 0x%x\n", FACTORYMEMORY_OFFSET + i * 4,
*ptr);
}
```

Console output:

CORTEX_M0P: TRACEID: 0x41c40000 -> 0x650725

CORTEX_M0P: DEVICEID: 0x41c40004 -> 0x2bb8802f

CORTEX_M0P: USERID: 0x41c40008 -> 0x80c7ae2d

CORTEX_M0P: BSLPIN_UART: 0x41c4000c -> 0x2150216

CORTEX_M0P: BSLPIN_I2C: 0x41c40010 -> 0x3020301

CORTEX_M0P: BSLPIN_INVOKE: 0x41c40014 -> 0x12a8

CORTEX_M0P: SRAMFLASH: 0x41c40018 -> 0x200080

CORTEX_M0P: PLLSTARTUP0_4_8MHZ: 0x41c4001c -> 0x41216

CORTEX_M0P: PLLSTARTUP1_4_8MHZ: 0x41c40020 -> 0x100021c

CORTEX_M0P: PLLSTARTUP0_8_16MHZ: 0x41c40024 -> 0x50b0f

CORTEX_M0P: PLLSTARTUP1_8_16MHZ: 0x41c40028 -> 0x1000110

CORTEX_M0P: PLLSTARTUP0_16_32MHZ: 0x41c4002c -> 0x5080d

CORTEX_M0P: PLLSTARTUP1_16_32MHZ: 0x41c40030 -> 0x1000110

CORTEX_M0P: PLLSTARTUP0_32_48MHZ: 0x41c40034 -> 0x5060a

CORTEX_M0P: PLLSTARTUP1_32_48MHZ: 0x41c40038 -> 0x1000110

CORTEX_M0P: TEMP_SENSE0: 0x41c4003c -> 0x31a

CORTEX_M0P: RESERVED0: 0x41c40040 -> 0x0

CORTEX_M0P: RESERVED1: 0x41c40044 -> 0x0

CORTEX_M0P: RESERVED2: 0x41c40048 -> 0x0

CORTEX_M0P: RESERVED3: 0x41c4004c -> 0x0

CORTEX_M0P: RESERVED4: 0x41c40050 -> 0x0

CORTEX_M0P: RESERVED5: 0x41c40054 -> 0x0

CORTEX_M0P: RESERVED6: 0x41c40058 -> 0x0

CORTEX_M0P: RESERVED7: 0x41c4005c -> 0x0

CORTEX_M0P: RESERVED8: 0x41c40060 -> 0x0

CORTEX_M0P: RESERVED9: 0x41c40064 -> 0x0

CORTEX_M0P: RESERVED10: 0x41c40068 -> 0x0

CORTEX_M0P: RESERVED11: 0x41c4006c -> 0x0

CORTEX_M0P: RESERVED12: 0x41c40070 -> 0x0

CORTEX_M0P: RESERVED13: 0x41c40074 -> 0x0

CORTEX_M0P: RESERVED14: 0x41c40078 -> 0x0

CORTEX_M0P: BOOTCRC: 0x41c4007c -> 0x2288472c

Appendix A – MCU Source Code

HelloWorld.c

```
#include "ti_msp_dl_config.h"

#include <stdio.h>

int main(void)
{
    printf("Hello World!!!\n");

    while (1) {
    }
}
```

BlinkLED.c

Part 1, DELAY macro set to 16000000

```
#include "ti_msp_dl_config.h"

/* This results in approximately 0.5s of delay assuming 32MHz CPU_CLK
*/

#define DELAY (16000000)

int main(void)
{
    /* Power on GPIO, initialize pins as digital outputs */
    SYSCFG_DL_init();

    /* Default: LED1 and LED3 ON, LED2 OFF */
```

```

DL_GPIO_clearPins(GPIO_LEDS_PORT, GPIO_LEDS_USER_LED_2_PIN);
DL_GPIO_setPins(GPIO_LEDS_PORT, GPIO_LEDS_USER_LED_1_PIN |
                                GPIO_LEDS_USER_LED_3_PIN |
                                GPIO_LEDS_USER_TEST_PIN);

while (1) {
    /*
    * Call togglePins API to flip the current value of LEDs 1-3.
This
    * API causes the corresponding HW bits to be flipped by the
GPIO HW
    * without need for additional R-M-W cycles by the processor.
    */
    delay_cycles(DELAY);
    DL_GPIO_togglePins(GPIO_LEDS_PORT,
                        GPIO_LEDS_USER_LED_1_PIN | GPIO_LEDS_USER_LED_2_PIN |
                        GPIO_LEDS_USER_LED_3_PIN | GPIO_LEDS_USER_TEST_PIN);
}
}

```

Part 2, DELAY macro set to 4000000

```
#include "ti_msp_dl_config.h"
```

```

/* This results in approximately 0.5s of delay assuming 32MHz CPU_CLK
*/

```

```
#define DELAY (4000000)
```

```
int main(void)
```

```

{
    /* Power on GPIO, initialize pins as digital outputs */
    SYSCFG_DL_init();

    /* Default: LED1 and LED3 ON, LED2 OFF */
    DL_GPIO_clearPins(GPIO_LEDS_PORT, GPIO_LEDS_USER_LED_2_PIN);
    DL_GPIO_setPins(GPIO_LEDS_PORT, GPIO_LEDS_USER_LED_1_PIN |
                                GPIO_LEDS_USER_LED_3_PIN |
                                GPIO_LEDS_USER_TEST_PIN);

    while (1) {
        /*
            * Call togglePins API to flip the current value of LEDs 1-3.
This
            * API causes the corresponding HW bits to be flipped by the
GPIO HW
            * without need for additional R-M-W cycles by the processor.
            */
        delay_cycles(DELAY);
        DL_GPIO_togglePins(GPIO_LEDS_PORT,
                            GPIO_LEDS_USER_LED_1_PIN | GPIO_LEDS_USER_LED_2_PIN |
                            GPIO_LEDS_USER_LED_3_PIN | GPIO_LEDS_USER_TEST_PIN);
    }
}

```

Access_MCU_Info.c

```
#include "ti_msp_dl_config.h"
```

```

#include <stdio.h>

#include <string.h>

int main(void)
{
    SYSCFG_DL_init();

    char FACTORYMEMORY_ACRONYM [100][100] = {"TRACEID", "DEVICEID",
"USERID", "BSLPIN_UART", "BSLPIN_I2C", "BSLPIN_INVOKE", "SRAMFLASH",
"PLLSTARTUP0_4_8MHZ",

    "PLLSTARTUP1_4_8MHZ", "PLLSTARTUP0_8_16MHZ",
"PLLSTARTUP1_8_16MHZ", "PLLSTARTUP0_16_32MHZ", "PLLSTARTUP1_16_32MHZ",
"PLLSTARTUP0_32_48MHZ",

    "PLLSTARTUP1_32_48MHZ", "TEMP_SENSE0", "RESERVED0", "RESERVED1",
"RESERVED2", "RESERVED3", "RESERVED4", "RESERVED5", "RESERVED6",

    "RESERVED7", "RESERVED8", "RESERVED9", "RESERVED10", "RESERVED11",
"RESERVED12", "RESERVED13", "RESERVED14", "BOOTCRC"};

    int FACTORYMEMORY_OFFSET = 0x41C40000;

    int *ptr;

    for (int i = 0; i < 32; i++){
        for (int j = 0; j < strlen(FACTORYMEMORY_ACRONYM[i]); j++){
            printf("%c", FACTORYMEMORY_ACRONYM[i][j]);
        }

        ptr = (int *) (FACTORYMEMORY_OFFSET + i * 4);
        printf(": 0x%x -> 0x%x\n", FACTORYMEMORY_OFFSET + i * 4,
*ptr);
    }
}

```

```
while (1) {  
  }  
}
```